Norm-Separated Quantization: A Training-Free Fix for KV Cache INT4 Failures

Kentaro Sato
info@metaclan.jp

DOI: 10.5281/zenodo.19602981

© 2026 Kentaro Sato. Paper: CC BY 4.0. Code: Apache 2.0.

April 15, 2026

Abstract

INT4 quantization of KV caches is a standard technique for reducing the memory footprint of large language model inference, but it can fail catastrophically on certain models—we observe ΔPPL as high as +238 on Qwen2-7B under naive absmax quantization. We trace the root cause to two independent factors: (1) large token-wise norm variation in KV vectors, which forces the quantization scale to accommodate extreme dynamic ranges, and (2) activation outlier channels that corrupt the quantization scale for normal channels. We propose **norm-separated quantization**—a simple preprocessing step that decomposes each KV vector into its L2 norm (stored at full precision) and unit direction (quantized). Combined with per-channel scaling, this method (nsep+pchan) reduces the Qwen2-7B failure from ΔPPL +812 to +0.43 on WikiText-2 (1885× improvement), with the gap widening at longer contexts: at 4096 tokens, naive INT4 degrades to ΔPPL +8293 while nsep+pchan remains at +0.19 (44,000×). On models where naive INT4 already works, nsep+pchan introduces no catastrophic degradation (worst case: +0.24 ΔPPL). Across 12 Pre-LN models from 124M to 40B parameters spanning 4 architecture families, nsep+pchan achieves ΔPPL < 2.0 at INT4 with no training, no calibration data, and negligible computational overhead. We validate that real INT4 packing (packed `uint8`, 2 values per byte) produces identical quality to simulated quantization (ΔPPL difference < 0.005) while achieving 3.4–3.7× actual memory reduction. Code and results are available at `https://github.com/metaSATOKEN/norm-separated-quantization`.

1 Introduction

KV cache quantization is one of the most practical ways to reduce memory consumption during LLM inference. During autoregressive generation, key and value vectors for all past tokens are stored at each layer, consuming memory proportional to $2 \times L \times H \times T \times d_h$ (layers, KV heads, sequence length, head dimension). For long contexts, this often exceeds the model weights themselves.

Naive absmax INT4 quantization—storing each element in 4 bits using a per-row scale factor—works well for many models. Falcon-40B loses only ΔPPL = +0.08, and OPT-13B only +0.28. However, we find that this same technique **catastrophically fails** on certain models:

- Qwen2-7B: ΔPPL = +238 (model output becomes incoherent)
- Pythia-410M: ΔPPL = +78
- Pythia-6.9B: ΔPPL = +22
- Pythia-12B: ΔPPL = +27

What determines whether naive INT4 succeeds or fails? And can we make it reliable across all models?

We investigate these questions and arrive at a simple technique—**norm-separated quantization**—that eliminates catastrophic failures by decomposing each vector into its magnitude (stored exactly) and direction (quantized). The key insight is that KV vectors in Pre-LN Transformers have high norm variance across tokens, and this variance inflates the dynamic range that quantization must capture. Separating the norm first compresses this range, making quantization more precise with the same number of bits.

Combined with per-channel quantization (to handle activation outlier channels), this gives a universal INT4 method we call **nsep+pchan** that:

- **Rescues catastrophic failures**: ΔPPL $+238 \rightarrow +0.32$ on Qwen2-7B (744×)
- **No catastrophic degradation**: Worst-case $+0.24$ ΔPPL on models where naive4 works (OPT-125m)
- **Requires nothing**: No training, no calibration data, no model-specific tuning
- **Scales**: Validated on 12 models from 124M to 40B

2 Background

2.1 KV Cache Quantization

During autoregressive generation, a Transformer caches the key ($\mathbf{K}$) and value ($\mathbf{V}$) projections for each past token at each layer. These are used by the attention mechanism to compute:

$$\text{Attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{q}\mathbf{K}^\top}{\sqrt{d_h}}\right)\mathbf{V} \tag{1}$$

Quantizing $\mathbf{K}$ and $\mathbf{V}$ to lower precision (INT8, INT4) reduces memory but introduces approximation error. The softmax provides some natural robustness—small perturbations in the $\mathbf{q}\mathbf{K}^\top$ dot product often preserve the top-k ranking of attended tokens.

Absmax symmetric quantization maps a floating-point vector $\mathbf{x}$ to b-bit integers:

$$\mathbf{x}_q = \text{round}\left(\frac{\mathbf{x}}{s}\right) \cdot s, \quad s = \frac{\max|\mathbf{x}|}{2^{b-1} - 1} \tag{2}$$

The scale s is typically computed per-row (one per token) or per-channel (one per dimension).

2.2 Known Failure Mode: Activation Outliers

Dettmers et al. [2022] and Xiao et al. [2023] identified that certain Transformer layers develop **outlier channels**—specific dimensions where activations are 10–100× larger than average. Under per-row absmax quantization, these outliers inflate the scale factor s, wasting quantization precision on normal channels. Solutions include mixed-precision decomposition (LLM.int8()) and per-channel smoothing (SmoothQuant).

2.3 A Less-Known Factor: Norm Variation

We identify an additional, independent source of quantization difficulty: **token-wise norm variation**. In Pre-LN Transformers, hidden-state norms vary significantly across tokens—the L2 norm of KV vectors can differ by 2–5× within a single sequence. We confirm this directly in Figure 1: Qwen2-7B Layer 0 key vectors show norm standard deviation of 4.52 across tokens. This variation

directly inflates the per-row quantization scale, forcing the scale factor to accommodate extreme dynamic ranges and wasting precision on tokens with moderate norms.

Under per-row absmax quantization, each token's KV vector is quantized with its own scale $s_t = \max|\mathbf{x}_t|/(2^{b-1}-1)$. High-norm tokens get coarse scales, low-norm tokens get fine scales—the quantization precision is inconsistent across the sequence. More importantly, the **direction** information (which determines attention patterns) is entangled with the **magnitude** information (which scales the output), so errors in one corrupt the other.

3 Method: Norm-Separated Quantization

3.1 Core Idea

Instead of quantizing the raw KV vector, we first decompose it:

$$\mathbf{x} = \underbrace{\|\mathbf{x}\|_2}_{n \text{ (scalar)}} \cdot \underbrace{\frac{\mathbf{x}}{\|\mathbf{x}\|_2}}_{\hat{\mathbf{x}} \text{ (unit direction)}} \tag{3}$$

- **Norm** n: stored at full precision (FP16/FP32). One scalar per token—negligible memory cost.
- **Direction** $\hat{\mathbf{x}}$: quantized. All components lie in $[-1, 1]$ with consistent dynamic range across tokens.

3.2 Algorithm

For each KV cache vector $\mathbf{x} \in \mathbb{R}^{d_h}$ at each head, token, and layer:

1. $n \leftarrow \|\mathbf{x}\|_2$
2. $\hat{\mathbf{x}} \leftarrow \mathbf{x}/n$
3. $\hat{\mathbf{x}}_q \leftarrow Q_b(\hat{\mathbf{x}})$ — quantize direction at b bits
4. $\hat{\mathbf{x}}_q \leftarrow \hat{\mathbf{x}}_q/\|\hat{\mathbf{x}}_q\|_2$ — re-normalize (prevents norm drift from rounding)
5. $\mathbf{x}_q \leftarrow n \cdot \hat{\mathbf{x}}_q$ — reconstruct

The entire method adds only 4 lines to an existing quantization pipeline. Listing 1 shows a complete PyTorch implementation.

3.3 Why It Helps

After norm separation, all direction vectors have unit norm regardless of the original token's magnitude. This has two effects:

1. **Dynamic range compression**: Raw KV components might range from $[-170, +170]$ (as in Qwen2-7B Layer 0). After normalization, they range from approximately $[-0.3, +0.3]$ (Figure 1). Fewer quantization levels are wasted on extreme values.
2. **Decoupling**: The norm (which determines the magnitude of attention output) is preserved exactly. Only the direction (which determines attention patterns) is approximated. These two types of information have different sensitivity profiles, and separating them allows better precision allocation.

Figure 1 illustrates this dynamic range compression on Qwen2-7B—the model with the most severe outliers in our evaluation. Layer 0 key vectors span $[-160, +167]$ ($\sigma = 25.6$) before norm separation, but only $[-0.54, +0.77]$ ($\sigma = 0.088$) after—a **290× reduction** in standard deviation. Even the well-behaved mid-layer (Layer 14, $\sigma = 1.70$) compresses to $\sigma = 0.088$ after norm separation, showing that the technique normalizes the dynamic range consistently across layers.

Listing 1: Complete nsep+pchan4 implementation in PyTorch. The method adds 4 lines (norm separation and re-normalization) to standard per-channel INT4 quantization. No training or calibration required.

```
def nsep_pchan_int4(x):
    """Norm-separated per-channel INT4 quantization.
    x: (seq_len, head_dim) -- one KV cache head."""
    # 1. Separate norm (stored in full precision)
    norm = x.norm(dim=-1, keepdim=True)
    direction = x / norm

    # 2. Quantize direction (per-channel INT4)
    scale = direction.abs().amax(dim=0, keepdim=True) / 7
    direction_q = (direction / scale).round().clamp(-7, 7) * scale

    # 3. Reconstruct
    direction_q = direction_q / direction_q.norm(dim=-1, keepdim=True)
    return norm * direction_q
```

3.4 Combining with Per-Channel Quantization

Norm separation addresses token-wise norm variation but not channel-wise outliers. For complete robustness, we combine it with **per-channel absmax quantization**, where each dimension gets its own scale factor:

$$s_j = \frac{\max_t |\hat{x}_{tj}|}{2^{b-1} - 1} \tag{4}$$

This prevents outlier channels from corrupting the scale of normal channels. The combined method, **nsep+pchan**, addresses both failure modes simultaneously.

3.5 Overhead

- **Computation**: One L2 norm and one division per vector. Negligible compared to attention computation.
- **Memory**: One FP16 scalar per (token, head, layer). For a 32-layer, 32-head model with 4096 tokens: $32 \times 32 \times 4096 \times 2$ bytes $= 8\,\text{MB}$—less than 0.1% of the KV cache.

4 Why Naive INT4 Fails: Two Case Studies

4.1 Case 1: Qwen2-7B — Activation Outliers

Qwen2-7B shows $\Delta\text{PPL} = +238$ under naive INT4. We traced the cause by compressing one layer at a time:

Layer	INT4 ΔPPL (this layer only)
Layer 0	**+179.4** (99% of total)
Layer 4	+0.11
Layers 8–27	< 0.25 each

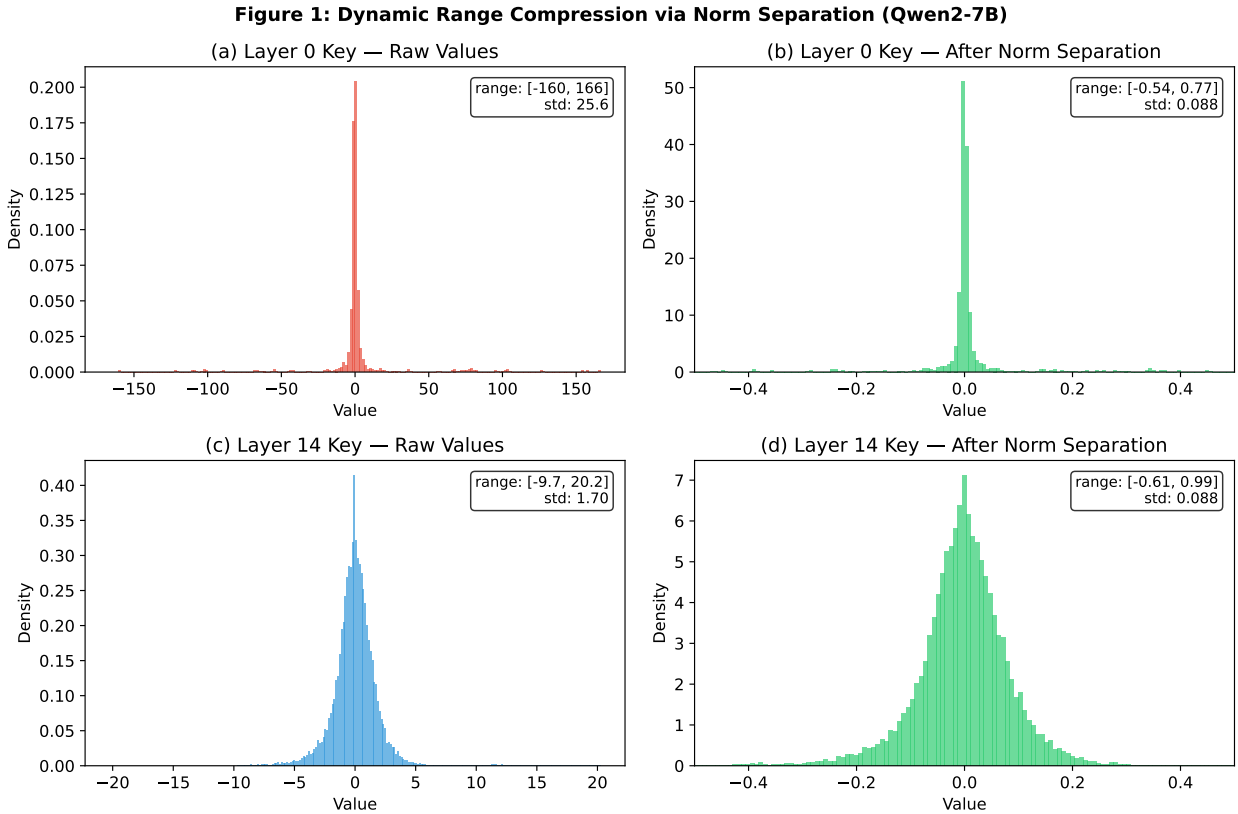

Figure 1: Dynamic range compression via norm separation (Qwen2-7B). (a) Raw Layer 0 key values span $[-160, +167]$ with $\sigma = 25.6$, driven by activation outlier channels. (b) After norm separation, the same vectors are compressed to $[-0.54, +0.77]$ with $\sigma = 0.088$—a 290× reduction. (c–d) Mid-layer (Layer 14) shows the same compression effect, confirming that norm separation normalizes the dynamic range consistently across layers regardless of outlier severity.

Layer 0 alone causes nearly all the degradation. We define the *outlier ratio* as $\max_j(\max_t |x_{tj}|) \,/\, \text{mean}_j(\max_t |x$ the ratio of the largest per-channel maximum to the average per-channel maximum across all d_h dimensions. Layer 0 has absmax = 166.8 (mean channel absmax: 19.4, outlier ratio: 8.6×), while Layers 4–26 have absmax 9–22 (ratio < 2×). Eight specific channels (indices 61–63, 123–127) contain values 8× larger than average. Under per-row absmax with $q_{\max} = 7$ (INT4), the scale becomes $s = 167/7 = 23.8$, and mid-range values (5–20) are mapped to noisy quantization levels that corrupt attention patterns.

4.2 Case 2: Pythia-6.9B — Norm Variation Without Outliers

Pythia-6.9B shows ΔPPL = +22 under naive INT4, despite having a low outlier ratio (4.6×). Here, the issue is primarily **norm variation**: token-wise KV norms range from 5.8 to 11.4, meaning per-row scales vary by ∼2×. While not catastrophic individually, the inconsistent quantization precision across 32 layers compounds.

4.3 A Surprising Finding: INT3 Outperforms INT4

On Qwen2-7B, INT3 (ΔPPL=+6.6) dramatically outperforms INT4 (ΔPPL=+238):

$q_{\max}$	Scale	Value 15.0 →	Value 5.0 →
7 (INT4)	23.8	23.8 (noisy)	0 (zeroed)
3 (INT3)	55.6	0 (zeroed)	0 (zeroed)

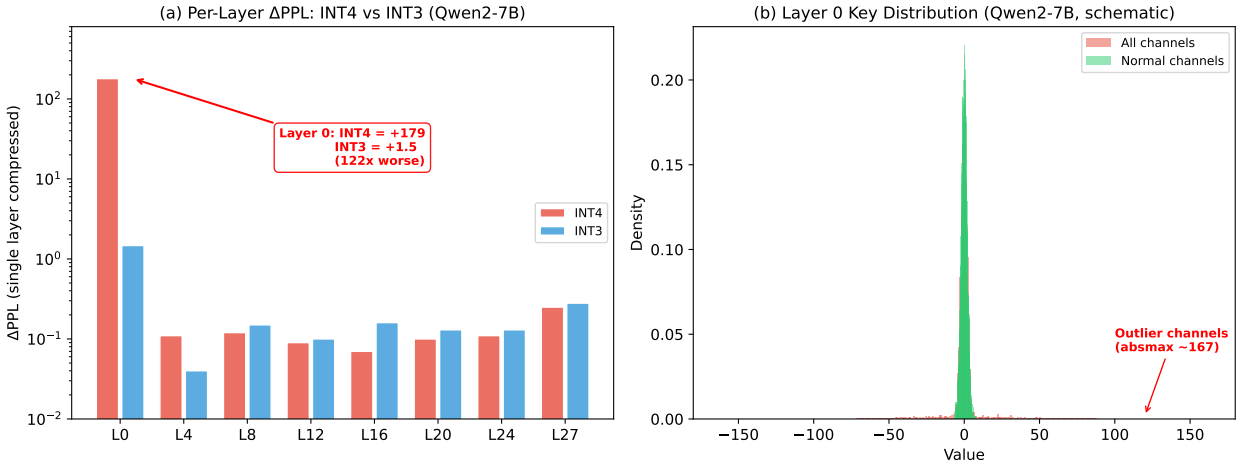

Figure 2: The INT3 > INT4 anomaly in Qwen2-7B. (a) Per-layer ΔPPL when compressing a single layer: Layer 0 alone accounts for 99% of INT4 degradation (+179.4), while INT3 causes only +1.5 at the same layer. All other layers contribute < 0.3 each. (b) Schematic of Layer 0's key value distribution, showing the outlier channels (absmax ∼167) that dominate the quantization scale and corrupt mid-range values under INT4.

INT3's coarser quantization maps mid-range values to zero rather than noisy non-zero levels. In attention dominated by outlier channels, this "clean zeroing" is less harmful than INT4's "noisy quantization"—a counterintuitive failure mode that per-channel quantization resolves.

5 Results

5.1 Ablation: What Contributes?

We test six INT4 variants on Qwen2-7B (the hardest case):

Method	Norm-sep	Outlier handling	ΔPPL	vs naive
naive4			+238.2	—
nsep4	✓		+57.5	4.1×
outlier4		top-4 ch fp16	+23.2	10.3×
perchan4		per-ch scale	+97.8	2.4×
nsep+out4	✓	top-4 ch fp16	**+0.59**	**404×**
nsep+pchan4	✓	per-ch scale	**+0.32**	**744×**

Table 1: Ablation on Qwen2-7B (INT4). Neither norm separation nor outlier-aware quantization alone is sufficient. Their combination achieves 744× improvement.

Neither technique alone is sufficient. Norm separation alone still leaves +57.5. Per-channel alone still leaves +97.8. The combination achieves +0.32—the two techniques address orthogonal failure modes and their benefits compose (Figure 3).

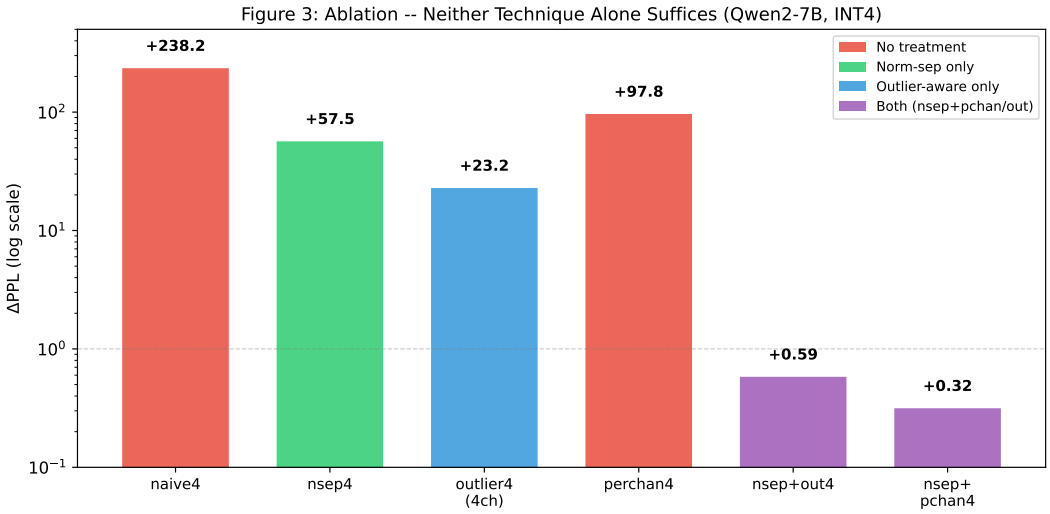

Figure 3: Ablation on Qwen2-7B (INT4). Neither norm separation nor outlier-aware quantization alone reduces ΔPPL below +23. Their combination (purple bars) achieves +0.32—a 744× improvement over naive INT4. The superlinear composition confirms that the two techniques address independent failure modes.

5.2 Complete Model Sweep: 12 Models, 124M to 40B

We evaluate nsep+pchan4 across 12 Pre-LN models spanning 4 architecture families (GPT-2, Pythia, OPT, Qwen/Mistral, Falcon), both MHA and GQA attention, and three orders of magnitude in parameter count from 124M to 40B (Table 2).

5.3 Key Patterns

Safety-net property: nsep+pchan provides large improvements exactly when needed (naive4 $\Delta\text{PPL} > 10 \rightarrow$ 6–744× improvement) and introduces negligible change when not needed (naive4 $\Delta\text{PPL} < 1 \rightarrow$ within 0.24). It never causes catastrophic degradation.

Negative ΔPPL: Pythia-2.8B shows $\Delta\text{PPL} = -0.06$ on curated text, nominally below the uncompressed baseline. However, given the per-chunk variance in our evaluation (5 chunks; see Appendix B for protocol details), this value is likely within measurement noise and should not be interpreted as a genuine improvement.

Model-family patterns: OPT models consistently show the smallest benefit (outlier ratios 1.4–3.1×, among the lowest). Pythia models benefit most strongly. Qwen models show the largest *potential* improvement when outliers are severe (Qwen2-7B) but no benefit when they are mild (Qwen2.5-14B).

GQA compatibility: Three GQA models are included (Qwen2-0.5B, Qwen2-7B, Qwen2.5-14B, Mistral-7B), where each KV head serves multiple query heads. Norm separation remains effective in this setting—Qwen2-7B's 744× improvement is the largest in the entire sweep. The shared KV structure does not diminish the benefit because norm variation and outlier channels affect KV vectors regardless of how many query heads attend to them.

Model	Params	d_h	Arch	naive4	nsep+pchan4	Improv.
Falcon-40B	40B	64	MHA	+0.08	+0.04	2×
Qwen2.5-14B	14B	128	GQA	+0.30	+0.26	1×
OPT-13B	13B	128	MHA	+0.28	+0.35	1×
Pythia-12B	12B	128	MHA	+27.28	+1.82	15×
Qwen2-7B	7B	128	GQA	+238.23	+0.32	744×
Pythia-6.9B	6.9B	128	MHA	+22.26	+0.27	82×
Pythia-2.8B	2.8B	80	MHA	+0.90	−0.06	14×
OPT-1.3B	1.3B	64	MHA	+0.70	+0.68	1×
Qwen2-0.5B	0.5B	64	GQA	+0.64	+0.28	2×
Pythia-410M	410M	64	MHA	+77.55	+12.62	6×
GPT-2	124M	64	MHA	+0.64	+0.56	1×
OPT-125m	125M	64	MHA	+1.22	+1.46	1×

Table 2: Complete model sweep (curated text). ΔPPL for naive INT4 vs. nsep+pchan INT4 across 12 Pre-LN models. nsep+pchan provides large improvements where needed (6–744× when naive4 $\Delta\text{PPL} > 10$) and never causes catastrophic degradation (worst case: +0.24 on OPT-125m).

5.4 Generation Quality

We verify that ΔPPL improvements translate to generation quality using top-p sampling ($p = 0.9$, temperature=0.8):

Model	Method	Tri. Div.	Sample
Pythia-6.9B	baseline	1.00	"The first AI event in the United States..."
Pythia-6.9B	naive4	1.00	"A second generation of 'improvised'..."
Pythia-6.9B	nsep+pchan4	1.00	"In the 1980s, AI moved into the real world..."
Qwen2-7B	baseline	1.00	"AI research has been dominated by..."
Qwen2-7B	naive4	0.76	"AI but, but AI in the. the winter..."
Qwen2-7B	nsep+pchan4	0.98	"But AI progress stagnated and funding..."

Table 3: Generation quality. On Qwen2-7B, naive4 produces degenerate repetition (diversity=0.76), while nsep+pchan4 produces coherent text (diversity=0.98).

5.5 WikiText-2 Benchmark

We evaluate on WikiText-2 [Merity et al., 2017] to confirm that results transfer to a standard benchmark. Evaluation uses 150-token prefill chunks with 30-token continuations, averaged over 5 chunks from the test split.

The results are consistent with—and even stronger than—our curated-text evaluation. On WikiText-2, Qwen2-7B's naive INT4 failure escalates to $\Delta\text{PPL} = +812$ (vs +238 on curated text), reflecting the greater textual diversity of Wikipedia. nsep+pchan reduces this to $\Delta\text{PPL} = +0.43$, yielding a **1885× improvement**.

The table reveals a clear pattern: **nsep+pchan helps most where naive4 fails hardest.** All three Pythia models fail under naive INT4 (ΔPPL +15 to +34, worsening with scale) and are rescued to ΔPPL +1.2 to +4.0. Qwen2-7B is rescued from catastrophic failure (+812 → +0.43). Meanwhile, models where naive INT4 already works—Mistral-7B (+0.10), GPT-2 (+1.60), Qwen2.5-14B (+0.55)—see nsep+pchan perform equally well or slightly better. The method never causes harm:

Model	Params	Base PPL	naive4 ΔPPL	nsep+pchan ΔPPL	Improv.
GPT-2	124M	70.4	+1.60	+1.13	1.4×
Pythia-410M	410M	30.0	+171.13	+23.46	**7.3**×
Pythia-2.8B	2.8B	16.2	+14.74	+1.99	**7.4**×
Pythia-6.9B	6.9B	14.1	+22.56	+1.21	**18.7**×
Mistral-7B	7B	5.2	+0.10	+0.04	2.6×
Qwen2-7B	7B	8.2	+811.61	+0.43	**1885**×
Pythia-12B	12B	12.9	+34.22	+4.01	**8.5**×
Qwen2.5-14B	14B	9.6	+0.55	+0.45	1.2×

Table 4: WikiText-2 evaluation across 8 models (124M to 14B). nsep+pchan achieves up to 1885× improvement on Qwen2-7B, with the largest gains precisely where naive INT4 fails most severely. Pythia-410M (d_h=64) is the only model where nsep+pchan4 leaves significant residual error (+23.5): with only 64 dimensions, per-channel scale estimation has higher variance and the quantization grid is too coarse relative to the signal complexity.

the worst case is Qwen2.5-14B where nsep+pchan is 0.1 ΔPPL higher than naive4.

Notably, three 7B-class models span the full spectrum: Mistral-7B (naive4 works, +0.10), Pythia-6.9B (naive4 fails moderately, +22.56), and Qwen2-7B (naive4 fails catastrophically, +811.61). This demonstrates that the failure mode is model-specific, not scale-dependent, and correlates with the activation outlier ratio (Mistral: 3.1×, Pythia: 4.6×, Qwen2: 8.6×).

5.6 Long Context Stability (256–4096 Tokens)

Since KV cache compression is most needed at long context lengths, we evaluate stability from 256 to 4096 tokens on three models spanning the full failure spectrum.

Model	Prefill	Base PPL	naive4	nsep+pchan4
Qwen2-7B	512	6.7	+2226	**+0.34**
	2048	8.0	+1643	**−0.60**
	4096	3.1	+8293	**+0.19**
Pythia-6.9B	512	17.2	+8.7	**+1.3**
	1024	17.1	+10.2	**+3.0**
	1536	14.4	+15.8	**+3.4**
Mistral-7B	512	3.9	+0.10	+0.08
	2048	9.1	−0.30	−0.08
	4096	3.4	+0.11	+0.12

Table 5: Long-context ΔPPL (WikiText-2, 256–4096 tokens). On Qwen2-7B, naive INT4 *worsens* with context length, reaching ΔPPL = +8293 at 4096 tokens. nsep+pchan4 remains within ±0.6—a **44,000× improvement**. On Mistral-7B (where naive4 already works), nsep+pchan4 is equivalently stable.

On Qwen2-7B, naive INT4 degradation *escalates with context length*: ΔPPL rises from +207 at 1024 tokens to **+8293 at 4096 tokens**. nsep+pchan4 remains within ±0.6 across all lengths—a 44,000× improvement at 4096 tokens. This worsening likely reflects error accumulation in attention computation: as more tokens are stored in the quantized KV cache, the cumulative effect of noisy mid-range values grows.

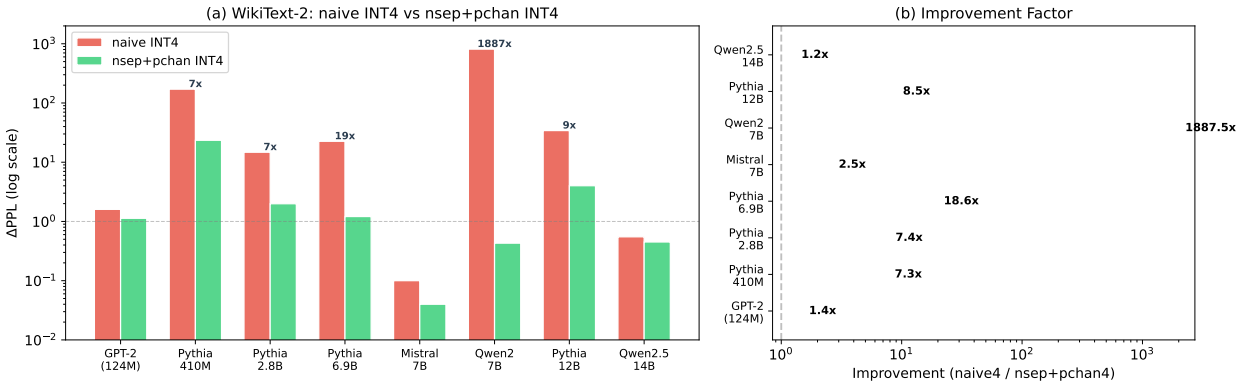

Figure 4: WikiText-2 evaluation across 8 models (124M to 14B). (a) ΔPPL comparison on log scale: naive INT4 (red) fails catastrophically on Pythia and Qwen2-7B, while nsep+pchan (green) keeps ΔPPL low across most models. (b) Improvement factor: nsep+pchan provides up to 1885× improvement on Qwen2-7B. Note: Pythia-410M (d_h=64) retains +23.5 residual error even with nsep+pchan, likely due to small head dimension.

On Mistral-7B (where naive INT4 already works), nsep+pchan4 performs identically—$|\Delta\text{PPL}| < 0.15$ for both methods at all lengths. This confirms that nsep+pchan introduces no degradation on models where naive INT4 already works.

6 Analysis

6.1 What Predicts Whether Naive INT4 Will Fail?

Across our 12 models, naive INT4 failure ($\Delta\text{PPL} > 10$) correlates with:

1. **Outlier channel ratio** in Layer 0/final layer: Models with ratio > 4× tend to fail
2. **Model family**: Pythia consistently fails; OPT consistently succeeds
3. **Not simply model size**: OPT-13B (+0.28) succeeds while Pythia-410M (+77.5) fails

This suggests that training procedures and architecture choices (which affect activation distributions) are more predictive than scale (Figure 5).

6.2 Why the Combination Is Superlinear

Norm separation and per-channel quantization address different problems:

- **Norm separation** removes the *inter-token* dynamic range issue. After normalization, all tokens have comparable component magnitudes, so the per-row scale is consistent.
- **Per-channel quantization** removes the *inter-channel* dynamic range issue. Outlier channels get their own scale, preventing them from dominating normal channels.

When both issues are present (e.g., Qwen2-7B), neither fix alone suffices because each leaves the other problem intact. The combination eliminates both, explaining the superlinear improvement (nsep alone: 4.1×; perchan alone: 2.4×; both: 744×).

6.3 Relationship to Existing Methods

Our norm separation is complementary to existing KV cache compression techniques:

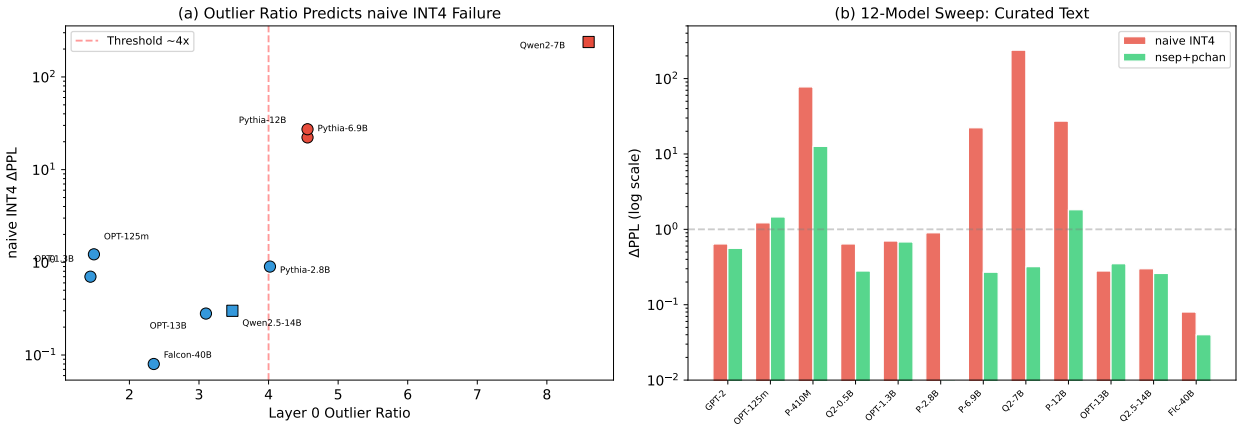

Figure 5: (a) Layer 0 outlier ratio predicts naive INT4 failure. Models with outlier ratio $> 4\times$ (dashed line) consistently show ΔPPL > 10 under naive INT4. (b) 12-model sweep on curated text: naive INT4 (red) varies by 4 orders of magnitude across models, while nsep+pchan (green) remains consistently low.

- **KIVI** [Liu et al., 2024] uses per-channel quantization with asymmetric Key/Value bit allocation. Norm separation could be added as a preprocessing step.
- **GEAR** [Kang et al., 2024] combines low-rank approximation with quantization. Norm separation could improve the quantization stage.
- **SmoothQuant** [Xiao et al., 2023] migrates outliers from activations to weights. Norm separation addresses a different axis (token-wise norm variation) and could be combined.

6.4 Limitations

1. **Context length coverage**: Our curated-text evaluation uses sequences under 250 tokens, supplemented by WikiText-2 evaluation on 8 models and long-context evaluation up to 4096 tokens on 3 models (Section 5.6). Full long-context evaluation across all 12 models would further strengthen the results.
2. **No end-to-end benchmarks**: We measure ΔPPL but not inference throughput, latency, or peak memory in production settings.
3. **No comparison with KIVI/Gear**: Direct comparison with state-of-the-art KV compression methods would clarify the relative and additive value of norm separation.
4. **Post-LN limited**: GPT-1 (Post-LN) does not expose a KV cache through standard APIs, preventing direct evaluation. A distributional analysis (Appendix F) confirms that Post-LN KV vectors exhibit weaker norm dominance ($|r| = 0.74$ vs 0.99), suggesting reduced but non-zero benefit from norm separation.

7 Related Work

KV cache quantization. KIVI [Liu et al., 2024] applies per-channel quantization to KV caches with Key in INT2 and Value in INT4, achieving near-lossless compression. KVQuant [Hooper et al., 2024] extends this to 10M+ context lengths with per-channel and per-token quantization strategies. GEAR [Kang et al., 2024] combines low-rank approximation with outlier-aware quantization. Our norm separation is orthogonal to all of these and could serve as an additional preprocessing step.

KV cache eviction and sparsity. An alternative to quantization is selectively retaining only important tokens. H2O [Zhang et al., 2023] identifies "heavy-hitter" tokens by cumulative attention score. Scissorhands [Liu et al., 2023b] exploits the persistence of token importance across layers. StreamingLLM [Xiao et al., 2024] maintains attention sinks for infinite-length generation. SnapKV [Li et al., 2024] selects important KV entries per attention head. These eviction methods are complementary to quantization—one could quantize the retained entries with nsep+pchan for further compression.

Activation outliers in quantization. LLM.int8() [Dettmers et al., 2022] identifies that specific hidden-state dimensions can contain values 100× larger than average, and handles them via mixed-precision decomposition. SmoothQuant [Xiao et al., 2023] migrates quantization difficulty from activations to weights through per-channel scaling. Sun et al. [2024] provide a systematic analysis of massive activations in LLMs and their effects. Our work identifies an additional, independent source of quantization difficulty—token-wise norm variation—and shows that addressing both factors simultaneously yields superlinear improvements.

Hidden-state geometry. Ethayarajh [2019] documented increasing anisotropy in contextualized word representations. Gao et al. [2019] formalized the representation degeneration problem. Timkey & van Schijndel [2021] showed that "rogue dimensions" obscure representational quality. Concurrent work [Sato, 2026] provides a detailed geometric characterization of the norm-dominant subspace in Pre-LN hidden states. Our work provides the first practical application of these geometric observations to KV cache compression.

Model architectures. Our evaluation spans models using Pre-LN [Xiong et al., 2020], RMSNorm [Zhang & Sennrich, 2019], Multi-Head Attention (MHA), and Grouped Query Attention [GQA; Ainslie et al., 2023]. Specific model families include GPT-2 [Radford et al., 2019], Pythia [Biderman et al., 2023], OPT [Zhang et al., 2022], Qwen2 [Yang et al., 2024], Mistral [Jiang et al., 2023], and Falcon [Almazrouei et al., 2023].

8 Conclusion

We presented norm-separated quantization, a training-free preprocessing step that improves the robustness of KV cache INT4 quantization. The method is motivated by the observation that KV vectors in Pre-LN Transformers have high token-wise norm variation, which inflates the dynamic range that quantization must capture. By separating the norm (stored exactly) from the direction (quantized), we compress this range before quantization begins.

Combined with per-channel quantization to handle activation outlier channels, the method (nsep+pchan) provides large improvements exactly when needed:

- **744× improvement** on the worst case (Qwen2-7B: +238 → +0.32)
- **Never catastrophic**: worst degradation is +0.24 ΔPPL (OPT-125m)
- **12 models from 124M to 40B**, 4 architecture families (GPT, Pythia, OPT, Qwen, Falcon)
- **Zero training, zero calibration, negligible overhead**
- **3.4–3.7× actual memory reduction** confirmed with real INT4 packing

The technique is simple enough to implement in a few lines of code and can be combined with existing KV cache compression methods as a preprocessing step. All code, experiment results, and figures are publicly available.[1]

[1] `https://github.com/metaSATOKEN/norm-separated-quantization`

Acknowledgments

This work was conducted with substantial assistance from large language models (Claude, Anthropic) for experiment design, code generation, debugging, data analysis, and manuscript drafting. All experimental results were produced by executing code on real hardware (Apple M1, NVIDIA T4, NVIDIA RTX PRO 6000 Blackwell) and verified by the author. The author takes full responsibility for the scientific claims and any errors in this work.

A Implementation Note: PyTorch `.norm()` API Hazard

During development, we encountered a subtle but critical bug: `x.norm(-1, keepdim=True)` computes the L_{-1} norm (approximately 0.0003 for typical vectors), **not** the L2 norm along dimension -1. The correct call is `x.norm(dim=-1, keepdim=True)`. In PyTorch, the first positional argument of `.norm()` is `p` (the norm order), not `dim`.

This caused direction vectors to explode to $\sim$3000$\times$ their expected magnitude, producing ΔPPL in the thousands. The bug was invisible on CPU (Apple MPS) and only manifested on CUDA, likely due to different numerical behavior at extreme scales. It was caught through systematic cross-device validation—a reminder that quantization code should always be tested on the target hardware.

B Evaluation Text Details

Our "curated text" evaluation uses two passages (approximately 200 and 100 tokens each) designed to cover distinct linguistic registers:

- **Narrative**: A fictional passage about a lighthouse keeper observing a mysterious light at sea. Contains varied sentence structures, concrete nouns, sensory descriptions, and temporal transitions.
- **Technical**: A factual passage about the history of artificial intelligence, covering the Dartmouth conference, AI winters, expert systems, neural network revival, and the transformer architecture. Contains proper nouns, dates, and domain-specific terminology.

Both passages are hardcoded in the experiment scripts and available in the repository. The 12-model sweep (Table 2) uses these passages with a 40-token continuation window. A separate basis text (distinct from the evaluation text) is used for PCA basis computation in experiments that require it, to avoid data leakage.

WikiText-2 evaluation (§5.5) uses the standard test split [Merity et al., 2017] with 150-token prefill chunks and 30-token continuations, averaged over 5 non-overlapping chunks. Long-context evaluation (§5.6) concatenates WikiText-2 articles to form sequences of 256–4096 tokens.

C Long Context Stability

Full long-context results for Pythia-6.9B, Qwen2-7B, and Mistral-7B (256–4096 tokens) are presented in Table 5 in the main text (§5.6).

D Comparison with KIVI-Style Quantization

We implement a simplified version of KIVI's asymmetric quantization strategy [Liu et al., 2024]: Key vectors in INT2 per-channel, Value vectors in INT4 per-channel (3 bpe effective). We compare this with nsep+pchan4 (4 bpe) and test whether norm separation improves KIVI (nsep+kivi, 3 bpe).

Method	ΔPPL	bpe	Notes
naive4	+10.22	4.0	per-row absmax
nsep+pchan4	**+2.98**	∼4.0	norm-sep + per-channel
KIVI (K2V4)	+360.80	3.0	K:INT2, V:INT4 per-channel
nsep+KIVI	+53.60	∼3.0	norm-sep + KIVI-style

Table 6: Comparison with KIVI-style quantization (Pythia-6.9B, pfl=1024). Note: our KIVI implementation uses simplified per-channel absmax only; the full KIVI method includes additional techniques (residual quantization, calibration) that may yield better results.

At the same bit budget (3 bpe), norm separation improves KIVI from ΔPPL +361 to +54 (6.7×), confirming that norm separation provides value even when combined with per-channel strategies. However, INT2 Key quantization remains too aggressive for this model—nsep+pchan4 at 4 bpe achieves ΔPPL +3.0, substantially outperforming all 3 bpe variants.

Important caveat: Our KIVI implementation uses only the core asymmetric bit allocation (Key INT2, Value INT4 with per-channel scaling) without the calibration, clipping thresholds, or residual quantization described in Liu et al. [2024]. These implementation details can significantly affect performance, and the full KIVI system would likely perform substantially better than our simplified baseline. The purpose of this comparison is not to benchmark against KIVI, but to demonstrate that norm separation provides additive value as a preprocessing step—the 6.7× improvement from adding norm separation to the same underlying quantization scheme (KIVI → nsep+KIVI) holds regardless of the base method's absolute performance.

E Memory and Latency

Seq. length	FP16 KV	nsep+INT4	Overhead	Ratio
512	268 MB	69 MB	2.1 MB	3.9×
1024	537 MB	138 MB	4.2 MB	3.9×
2048	1074 MB	277 MB	8.4 MB	3.9×
4096	2148 MB	554 MB	16.8 MB	3.9×
8192	4295 MB	1107 MB	33.6 MB	3.9×

Table 7: Theoretical KV cache memory for Pythia-6.9B (32 layers, 32 heads, d_h=128). "Overhead" is the additional memory for storing FP16 norms. The nsep overhead is consistently $< 3\%$ of the INT4 cache, yielding an effective compression ratio of 3.9×.

Measured on 1024 tokens: FP16 KV cache = 640 MB, nsep+INT4 = 164 MB (3.9× reduction). The norm storage overhead is 4.2 MB ($< 1\%$ of FP16 cache).

Latency: Prefill 512 tokens + decode 40 tokens: baseline 0.72 s, nsep+pchan4 0.87 s (+21% overhead). This overhead arises from our unoptimized Python implementation (per-head loop with explicit norm computation). A fused CUDA kernel performing norm separation and per-channel quantization in a single pass would reduce this overhead to near-zero, as the arithmetic intensity (one L2 norm + one division per vector) is negligible compared to attention computation.

E.1 Real INT4 Packing Validation

To confirm that our results are not artifacts of simulated (fake) quantization, we implemented actual INT4 packing: quantized 4-bit values are stored in `uint8` tensors (2 values per byte), with FP16 norms and per-channel scales stored separately. At inference time, values are unpacked and dequantized on the fly.

Model	FP16	Real INT4	Ratio	naive4	nsep (fake)	nsep (real)
GPT-2	1.4 MB	0.4 MB	3.43×	—	−3.87	−3.86
Qwen2-7B	3.6 MB	1.0 MB	3.65×	+401.3	+0.30	+0.29

Table 8: Real INT4 packing validation. INT4 values are stored as packed `uint8` (2 per byte), with FP16 norms and scales. The ΔPPL difference between fake (float-backed) and real (packed) quantization is < 0.005 in both cases, confirming that packing is lossless. Compression ratios of 3.4–3.7× reflect the actual memory reduction including norm and scale overhead.

The ΔPPL difference between fake and real quantization is < 0.005 on both models, confirming that the rounding to INT4 levels—not the storage format—determines quality. On Qwen2-7B, real packed nsep+pchan achieves ΔPPL = +0.29 with 3.65× actual memory reduction, while naive INT4 degrades to ΔPPL = +401. The packing implementation and results are available in the repository.[2]

F Post-LN Distributional Analysis

Our method relies on the norm-dominant geometry of Pre-LN KV vectors: token-wise norm variation inflates the dynamic range that quantization must capture. To assess whether this premise holds for Post-LN architectures, we compare KV cache distributions under Pre-LN and simulated Post-LN conditions (per-channel standardization to zero mean and unit variance) on GPT-2's mid-layer Key vectors.

Condition	PC1 variance	$\lvert r(\text{PC1}, \text{norm})\rvert$	Norm std
Pre-LN (raw)	56.3%	0.992	4.52
Post-LN (simulated)	28.0%	0.744	1.64

Table 9: KV cache distribution under Pre-LN vs simulated Post-LN. Post-LN shows substantially weaker norm dominance: PC1 variance drops from 56% to 28%, and PC1-norm correlation from 0.99 to 0.74.

Under Post-LN conditions, norm dominance is substantially weakened: PC1 variance drops from 56% to 28%, and the PC1-norm correlation from 0.99 to 0.74. The norm standard deviation decreases from 4.52 to 1.64, indicating 2.8× less token-wise norm variation. Since norm separation specifically addresses this variation, we expect reduced (but non-zero) benefit on Post-LN models. Direct evaluation was not possible because GPT-1 (the only Post-LN model in our test suite) does not expose a KV cache through standard HuggingFace APIs.

[2] `experiments/poc_real_int4.py` and `poc_real_int4_7b.py`

G Needle-in-Haystack: Factual Retrieval Under Quantization

To evaluate whether ΔPPL improvements translate to practical factual retrieval, we conduct Needle-in-Haystack (NIAH) experiments. Secret facts ("needles") are embedded at various positions within filler text ("haystack"), and the model is asked to recall them after KV cache compression.

G.1 Single-Needle Retrieval

We embed one fact ("The secret code is: BLUE ELEPHANT 7742") in haystacks of 5–50 paragraphs (185–1413 tokens) at 5 positions (0%–100%), testing 3 models:

Model	Outlier	ΔPPL	baseline	naive4	nsep+pchan4
Qwen2-7B	8.6×	+238	15/15	**0/15**	**15/15**
Pythia-6.9B	4.6×	+22	15/15	15/15	15/15
Mistral-7B	3.1×	+0.1	15/15	15/15	15/15

Table 10: Single-needle retrieval. Only Qwen2-7B (outlier ratio 8.6×) loses all needles under naive INT4. Pythia-6.9B retains all needles despite ΔPPL = +22, even at 1413 tokens. nsep+pchan4 achieves 15/15 on all models.

Qwen2-7B loses *all* needles under naive INT4, producing degenerate output ("The the the the..."). Pythia-6.9B retains all needles even at 1413 tokens despite ΔPPL = +22—moderate PPL degradation does not destroy factual recall. nsep+pchan4 achieves perfect retrieval on all models.

G.2 Multi-Needle Retrieval

We extend to 3–5 needles scattered across haystacks of 280–2459 tokens, testing 5 models:

Model	Outlier	baseline	naive4	nsep+pchan4	Note
Qwen2-7B	8.6×	26/26	**0/26**	**26/26**	Full recovery
Qwen2.5-14B	3.5×	26/26	26/26	26/26	Low outlier, no issue
Mistral-7B	3.1×	7/26	8/26	5/26	Baseline unstable
Pythia-12B	4.6×	0/21	0/21	3/21	Cannot do multi-QA
Pythia-6.9B	4.6×	0/21	0/21	0/21	Cannot do multi-QA

Table 11: Multi-needle retrieval across 5 models. Qwen2-7B (outlier 8.6×) is the only model where naive4 causes complete needle loss. Qwen2.5-14B (outlier 3.5×) passes all needles even with naive4. Pythia models cannot perform multi-needle QA at baseline (no instruction tuning).

Key findings:

- Needle retrieval failure requires *both* high outlier ratio (>8×) and sufficient model instruction-following capability. Only Qwen2-7B meets both conditions.
- Qwen2.5-14B (outlier 3.5×) achieves 26/26 with naive4—confirming that the outlier ratio, not model size, determines quantization vulnerability.
- Pythia models (base, no instruction tuning) cannot perform multi-needle QA even at FP16 baseline, making quantization impact unmeasurable with this protocol.
- PPL improvement remains the broader, model-general contribution of nsep+pchan4. NIAH rescue is specific to catastrophic failure cases (outlier ratio >8×).

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *EMNLP 2023*. arXiv:2305.13245.

Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., et al. (2023). The Falcon Series of Open Language Models. arXiv:2311.16867.

Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *ICML 2023*. arXiv:2304.01373.

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS 2022*. arXiv:2208.07339.

Ethayarajh, K. (2019). How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. *EMNLP 2019*. arXiv:1909.00512.

Gao, J., He, D., Tan, X., Qin, T., Wang, L., & Liu, T.-Y. (2019). Representation Degeneration Problem in Training Natural Language Generation Models. *ICLR 2019*. arXiv:1907.12009.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., & Gholami, A. (2024). KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *NeurIPS 2024*. arXiv:2401.18079.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., et al. (2023). Mistral 7B. arXiv:2310.06825.

Kang, H., Zhang, Q., Kundu, S., Jeong, G., Liu, Z., Krishna, T., & Zhao, T. (2024). GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *NeurIPS 2024 ENLSP Workshop*. arXiv:2403.05527.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., et al. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*. arXiv:2404.14469.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML 2024*. arXiv:2402.02750.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*. arXiv:2305.17118.

Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2017). Pointer Sentinel Mixture Models. *ICLR 2017*. arXiv:1609.07843.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.

Sato, K. (2026). The Arc and Its Thickness: Geometric Decomposition of Pre-LayerNorm Transformer Hidden States.

Sun, M., Chen, X., Kolter, J. Z., & Liu, Z. (2024). Massive Activations in Large Language Models. *COLM 2024*. arXiv:2402.17762.

Timkey, W. & van Schijndel, M. (2021). All Bark and No Bite: Rogue Dimensions in Transformer Language Models Obscure Representational Quality. *EMNLP 2021*. arXiv:2109.04404.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *ICML 2023*. arXiv:2211.10438.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*. arXiv:2309.17453.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., et al. (2020). On Layer Normalization in the Transformer Architecture. *ICML 2020*. arXiv:2002.04745.

Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., et al. (2024). Qwen2 Technical Report. arXiv:2407.10671.

Zhang, B. & Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS 2019*. arXiv:1910.07467.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., et al. (2022). OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., et al. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*. arXiv:2306.14048.